CS5182 Computer Graphics

Course Project Report

Benchmarking PointNet++ and DGCNN for 3D Point Cloud Classification

Students: ZHUANG Yi (58800510), Leader
ZHU Chunjin (59845469)
Xia Junxia (59326842)

Course: CS5182 Computer Graphics
Semester: 2025/2026 Semester B
Topic: 3D point cloud classification benchmark
Methods: PointNet++ and DGCNN
Datasets: ModelNet40 and ScanObjectNN

April 2026

Abstract

This project studies 3D point cloud object classification in a reproducible PyTorch pipeline. We use PointNet++ as the baseline method and DGCNN as the comparison method, and evaluate them on two datasets: ModelNet40 and ScanObjectNN. To keep the report and presentation consistent, we use four unified experiment names throughout the project: *PointNet++ on ModelNet40 fast*, *DGCNN on ModelNet40 fast*, *PointNet++ on ScanObjectNN lightweight*, and *DGCNN on ScanObjectNN lightweight*. Our local experiments show that the two methods achieve very similar performance on ModelNet40 under the matched fast setting. Under our lightweight ScanObjectNN setting, PointNet++ performs better than DGCNN. All reported results are lower than the stronger numbers in the original papers, which is expected because our runs use a smaller local-compute setup, CPU execution, fewer input points, and reduced training on ScanObjectNN. The main contribution of this course project is therefore not a novel method. Instead, it is a complete and honest benchmark-and-analysis pipeline that satisfies the course requirements of implementation, training, comparison with the literature, second-dataset analysis, and discussion of limitations.

Keywords: point cloud classification; PointNet++; DGCNN; ModelNet40; ScanObjectNN

1 Introduction

Deep learning on point clouds is an important topic in computer graphics and 3D vision because real 3D sensors naturally produce unordered point sets rather than regular image grids or clean meshes. Point cloud classification is a representative entry task: given a set of 3D points sampled from one object, the model predicts the semantic category of that object. This project follows the CS5182 course requirement of reproducing a learning-based graphics method, training it, comparing it with another method, evaluating it on a second dataset, and analyzing its limitations.

Our project is centered on two representative point-based classifiers. PointNet++ [2] extends PointNet [1] by learning hierarchical local features through set abstraction. DGCNN [3] instead builds dynamic local graphs and applies EdgeConv layers to capture neighborhood relations. We implemented and organized both methods in one modular benchmark codebase, then compared them on a cleaner synthetic dataset and a more difficult real-world dataset.

2 Task Definition and Motivation

2.1 Task Definition

The task is **3D point cloud object classification**. The input is a point cloud

$$X \in \mathbb{R}^{N\times 3},$$

where each row is a 3D point with (x, y, z) coordinates. In the four reported experiments, $N = 512$ points are sampled from each object instance. The output is a class label

$$y \in \{1, \ldots, C\},$$

where $C = 40$ for ModelNet40 and $C = 15$ for ScanObjectNN.

2.2 Motivation

The project is motivated by two practical questions. First, can we build a clean and reproducible local training pipeline for point cloud classification with two representative deep models? Second, how much does model behavior change when we move from a synthetic benchmark to a real-world

dataset with clutter, partial observations, and noisier geometry?

These questions align well with the course rubric. ModelNet40 provides a standard benchmark for controlled comparison, while ScanObjectNN provides a more realistic test of generality. Together they allow us to analyze both implementation correctness and practical limitations.

2.3 Technical Challenges

Three technical challenges are central in this task.

- Point clouds are unordered, so the model must be permutation-invariant.
- Local geometric structure matters, but unlike image grids, point clouds do not have fixed neighborhoods.
- Real-world scans are incomplete, noisy, and may contain clutter or partial views, which makes the second dataset substantially harder than the synthetic benchmark.

3 Dataset and Preprocessing

3.1 ModelNet40

ModelNet40 was introduced as part of the ModelNet dataset [5]. In the local dataset files used by this project, the official split contains 9,840 training shapes and 2,468 test shapes. It is a clean CAD-model benchmark with 40 object categories and is widely used for point cloud classification. In our project it serves as the main baseline benchmark.

3.2 ScanObjectNN

ScanObjectNN is a real-world object benchmark built from scanned indoor scenes [4]. The repository uses the `pb_t50_rs` variant with 15 classes. The local `h5` files used by the project contain 11,416 training samples and 2,882 test samples for this variant. Compared with ModelNet40, ScanObjectNN is harder because objects are derived from real scans and are affected by incomplete reconstruction, non-uniform density, occlusion, partial observation, and scene clutter.

For the two experiments reported as *ScanObjectNN lightweight*, we did **not** use the full train/test split. To keep same-day local experiments practical, we used a reduced subset with 4,096 training samples and 1,024 test samples, 512 points per object, and 6 training epochs. This setting should be interpreted honestly as a lightweight local comparison rather than as a full-scale benchmark reproduction.

3.3 Preprocessing Pipeline

The same general preprocessing pipeline is applied through the dataset loaders in the repository.

- Fixed-size point sampling: each object is sampled to 512 points for the four reported experiments.
- Normalization: each point cloud is centered to zero mean and scaled to unit radius.
- Training-time augmentation: random point dropout, random scaling, random shifting, and Gaussian jitter are enabled.
- Upright-axis rotation augmentation is implemented in the pipeline but disabled in the reported runs.

The project only uses xyz coordinates as input channels in the reported experiments; normals are not used in the final results.

Table 1: Dataset summary used in this report.

Dataset	Classes	Local Train Split	Local Test Split	Notes
ModelNet40	40	9,840	2,468	Clean CAD benchmark used as the main baseline dataset.
ScanObjectNN (`pb_t50_rs`)	15	11,416	2,882	Real-world benchmark with clutter, partiality, and noisier geometry.

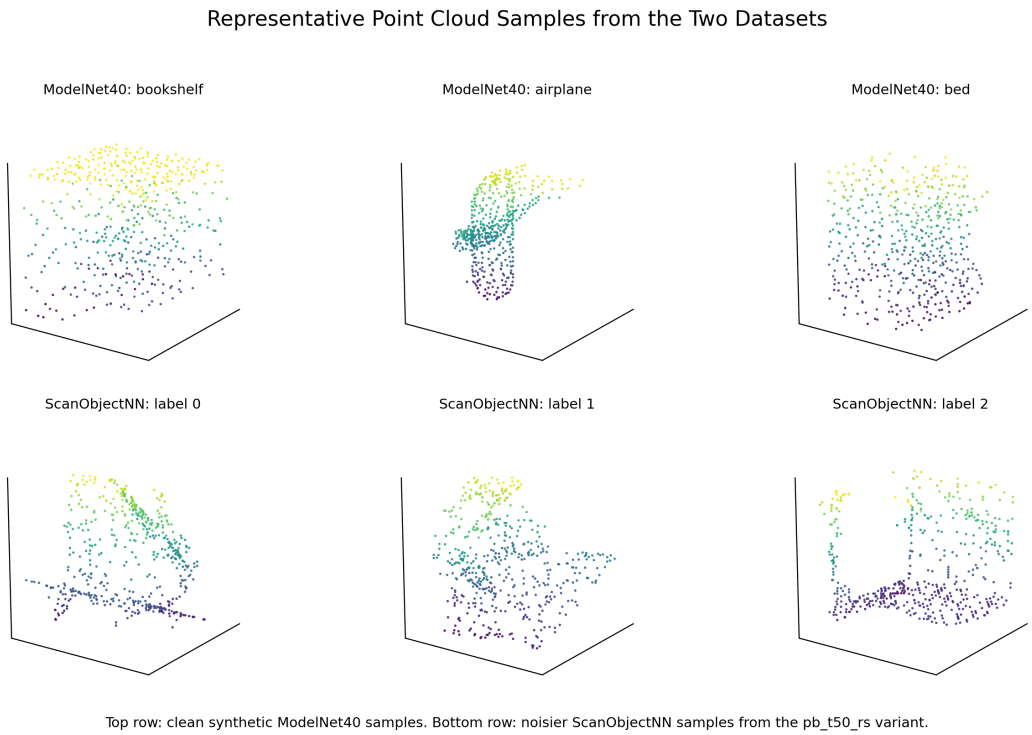

Figure 1: Representative point cloud samples from the two datasets. ModelNet40 samples are clean and complete, while ScanObjectNN samples are visibly noisier and more partial. This visual difference helps explain why the second dataset is harder.

4 Methods

4.1 PointNet++

PointNet++ [2] extends PointNet by introducing hierarchical set abstraction. The implementation in this repository is a single-scale grouping classifier. It progressively samples points and aggregates local neighborhoods into higher-level features. In our code, the network uses three set abstraction stages: local regions are first sampled at 512 centroids, then 128 higher-level regions, and finally aggregated into one global feature vector. The classification head is a multi-layer perceptron with dropout and batch normalization.

The strength of PointNet++ is that it explicitly learns local geometric structure while still operating directly on point sets. This makes it a strong baseline for shape classification.

4.2 DGCNN

DGCNN [3] models local geometry through dynamic graphs and EdgeConv layers. Instead of using a fixed hierarchy only in Euclidean space, it repeatedly constructs k-nearest-neighbor relations and updates them in feature space across layers. This allows the model to capture both local geometric relations and broader semantic affinity.

In our implementation, DGCNN uses four EdgeConv blocks, feature fusion, global max/average pooling, and a final classifier. The ModelNet40 fast configuration explicitly sets $k = 20$, embedding dimension 1024, and classifier dropout 0.5.

4.3 Why These Two Methods

The two methods provide a meaningful comparison under the course requirement of implementing and comparing another state-of-the-art method for the same task.

- PointNet++ is the baseline and represents hierarchical point-set abstraction.
- DGCNN is the comparison method and represents graph-based local feature learning.

This pairing lets us compare two different ways of handling local neighborhood structure on point clouds.

5 Experimental Setup

5.1 Environment and Dependencies

The codebase is written in PyTorch. According to the repository requirements, the main dependencies are `torch`, `numpy`, `PyYAML`, `h5py`, and `tqdm`. The reported experiment configs explicitly set the runtime device to `cpu`. This is an important practical constraint because the course PDF strongly recommends GPU training, while our report-quality runs were prepared under CPU-oriented local-compute settings.

5.2 Executed Pipeline and Scripts

To satisfy the "run demo code" and pipeline-understanding part of the course requirement, we organized the project into a reusable workflow rather than a single script. The core scripts are:

- `scripts/train.py`: load config, build data/model, train the classifier, and save checkpoints and history;
- `scripts/evaluate.py`: load the best checkpoint and compute final evaluation accuracy;
- `scripts/visualize.py`: generate training-curve figures from saved history files.

The repository also includes smoke-test scripts for ModelNet40, ScanObjectNN, and PointNet++ forward verification. These scripts were useful for environment setup, path checking, and debugging, but their outputs are **not** used as report results.

5.3 Step-by-Step Reproduction Notes

Beyond listing the scripts, the execution chain of the project can be described step by step as follows.

1. **Prepare the experiment config.** A YAML file specifies the dataset, model, point count, batch size, epoch count, optimizer settings, runtime device, and output folders.
2. **Sanity-check the data path.** Smoke-test scripts confirm that the dataset can be loaded and that the point tensors have the expected shape.
3. **Train the model.** The training script builds the dataloaders and network, applies cross-entropy loss with Adam, and saves `latest.pt`, `best.pt`, and `train_history.json`.
4. **Evaluate the best checkpoint.** The evaluation script reloads the saved best checkpoint and computes the final held-out accuracy.
5. **Generate visual summaries.** The visualization script converts saved history files into training-curve figures for the report and presentation.

Representative command patterns used in the project are:

- `python scripts/smoke_test_modelnet40.py -config configs/pointnet2_modelnet40_debug.yaml`
- `python scripts/train.py -config configs/pointnet2_modelnet40_fast.yaml`
- `python scripts/evaluate.py -config configs/pointnet2_modelnet40_fast.yaml`
- `python scripts/visualize.py -config configs/pointnet2_modelnet40_fast.yaml`

Figure 2 summarizes how these steps connect from input configuration to final report artifacts.

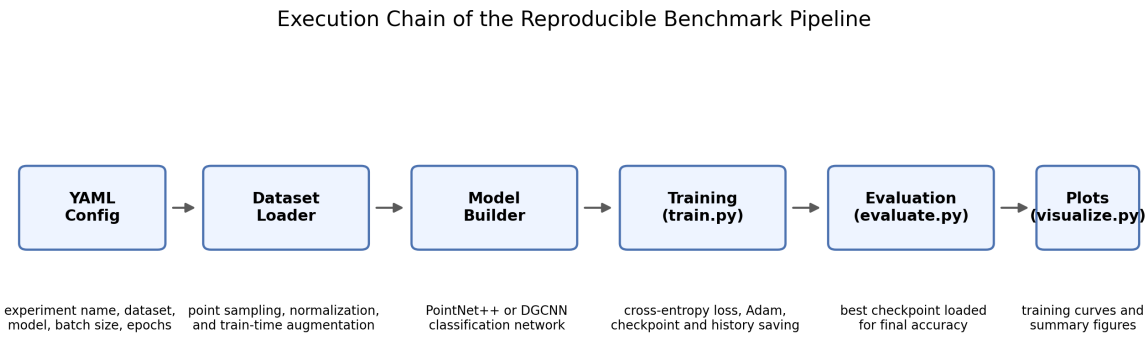

Figure 2: Execution chain of the benchmark pipeline. The figure complements the script list by showing how configuration, data loading, model building, training, evaluation, and visualization connect to each other.

5.4 Training Configuration

All four reported experiments use seed 42, Adam optimizer, cross-entropy loss, learning rate 10^{-3}, weight decay 10^{-4}, and batch size 16. The four final experiments are:

- **PointNet++ on ModelNet40 fast**: 512 points, 25 epochs, full ModelNet40 split, CPU.
- **DGCNN on ModelNet40 fast**: 512 points, 25 epochs, full ModelNet40 split, CPU.
- **PointNet++ on ScanObjectNN lightweight**: 512 points, 6 epochs, reduced subset (4,096 train / 1,024 test), CPU.
- **DGCNN on ScanObjectNN lightweight**: 512 points, 6 epochs, reduced subset (4,096 train / 1,024 test), CPU.

Table 2: Configuration summary of the four reported experiments.

Experiment	Points	Epochs	Batch Size	Device	Split Scope
PointNet++ on ModelNet40 fast	512	25	16	CPU	Full local ModelNet40 train/test split.
DGCNN on ModelNet40 fast	512	25	16	CPU	Full local ModelNet40 train/test split.
PointNet++ on ScanObjectNN lightweight	512	6	16	CPU	Reduced subset: 4,096 train / 1,024 test.
DGCNN on ScanObjectNN lightweight	512	6	16	CPU	Reduced subset: 4,096 train / 1,024 test.

5.5 Evaluation Protocol

The main reported metric is classification accuracy. For each experiment, we report final evaluation accuracy from the saved evaluation JSON, together with best validation accuracy and best epoch from the saved training history. Smoke-test, debug, and verify artifacts are intentionally excluded from the main results table.

One implementation detail should be stated explicitly. In the current scaffold, the trainer monitors accuracy on the held-out split built with `split='test'` during training, so the recorded `val_acc` in `train_history.json` is computed on the same held-out split that is later used by the final evaluation script. The final "test accuracy" is therefore a separate evaluation pass on the same held-out split using the saved best checkpoint. This explains why the best validation accuracy and final test accuracy can be identical in several experiments.

6 Results and Comparison

6.1 Main Results

Table 3 summarizes the four target experiments. Under the matched *fast* setting on ModelNet40, PointNet++ and DGCNN are extremely close: 87.24% vs. 87.03%. Under the *lightweight* ScanObjectNN setting, PointNet++ reaches 74.12% while DGCNN reaches 66.02%.

The interpretation of the second dataset must remain careful. These are not full ScanObjectNN benchmark runs; they are lightweight local-compute experiments using a reduced subset and only 6 epochs. Therefore, the ModelNet40 comparison is the cleaner method-to-method comparison, while the ScanObjectNN results mainly support analysis of cross-dataset difficulty and local-compute behavior.

Table 3: Main results of the four target experiments. Only final report-quality artifacts are included.

Experiment	Test Accuracy (%)	Best Val. Accuracy (%)	Best Epoch
PointNet++ on ModelNet40 fast	87.24	87.24	25
DGCNN on ModelNet40 fast	87.03	87.03	25
PointNet++ on ScanObjectNN lightweight	74.12	74.22	6
DGCNN on ScanObjectNN lightweight	66.02	66.02	5

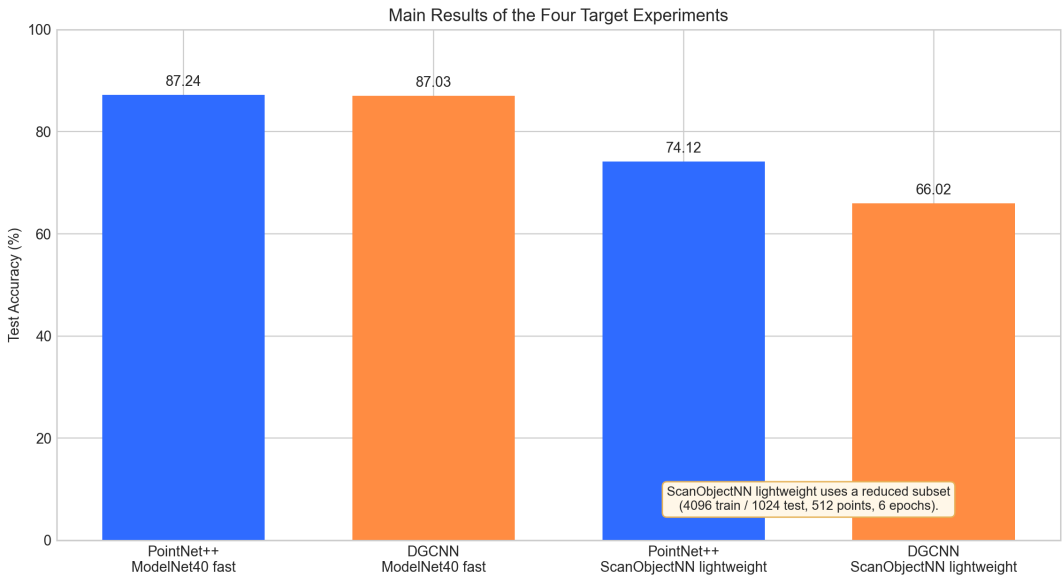

Figure 3: Accuracy comparison of the four target experiments. The ScanObjectNN lightweight results use the reduced subset setting and should be interpreted accordingly.

6.2 Training Curves

The training curves in Figure 4 show that both ModelNet40 runs improve steadily across 25 epochs, with the best epoch appearing at the end of training for both methods. For the lightweight ScanObjectNN runs, PointNet++ continues to improve until epoch 6, while DGCNN peaks earlier at epoch 5 and then ends slightly lower on validation accuracy. This trend is consistent with the final evaluation gap in our local lightweight setting.

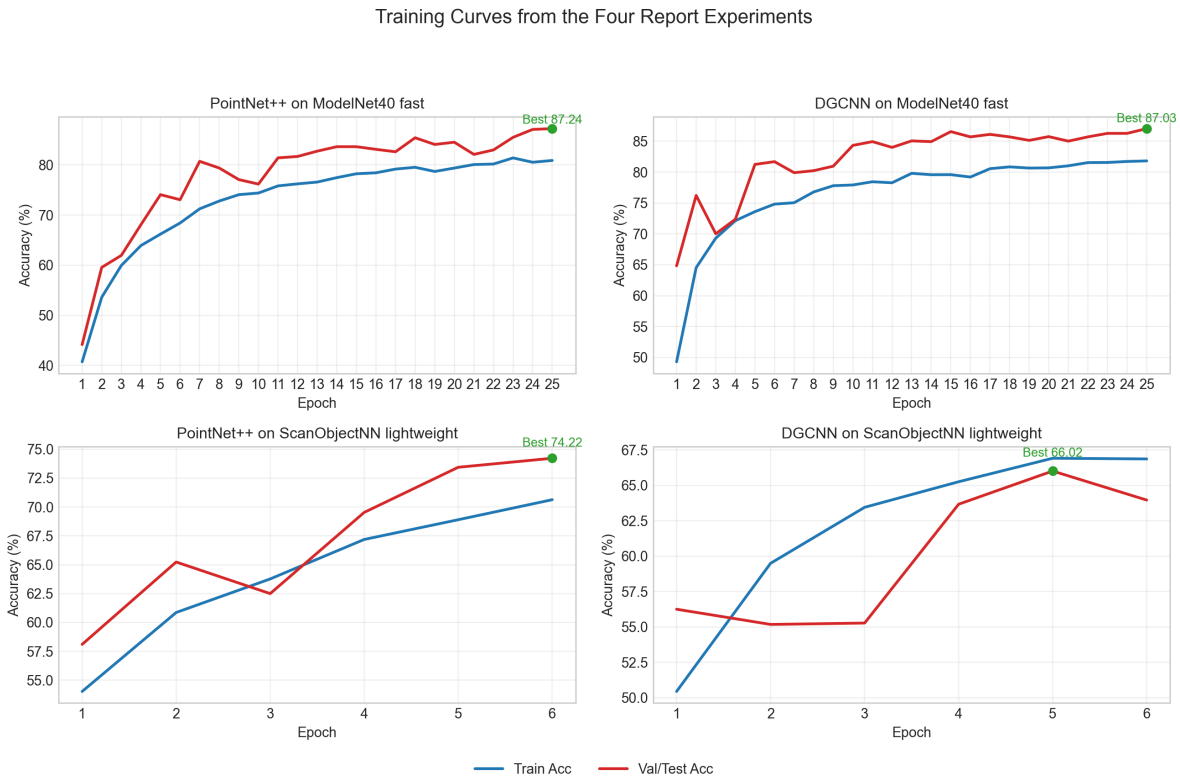

Figure 4: Training and validation/test accuracy curves generated from the saved `train_history.json` files of the four report experiments.

6.3 Point-Cloud Visualized Results

To supplement the ML-style plots with more task-specific visual evidence, Figure 5 shows qualitative prediction examples from *PointNet++ on ScanObjectNN lightweight*. The top row contains correctly classified examples, while the bottom row contains misclassified examples from the held-out split.

These examples do not replace a full failure benchmark, but they make two important points more concrete. First, ScanObjectNN objects can already look ambiguous or incomplete after subsampling to 512 points. Second, the misclassified examples often have less stable global shape cues than the correctly classified ones. This visual evidence is consistent with the dataset-difficulty discussion in the ScanObjectNN paper and with our own limitation analysis.

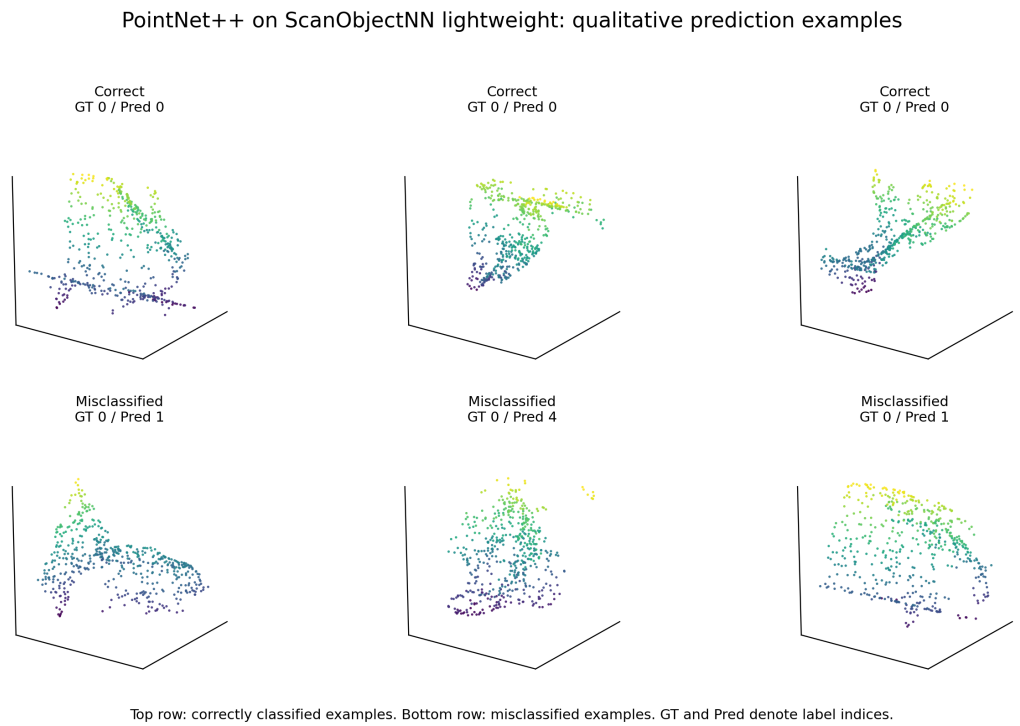

Figure 5: Qualitative prediction examples for PointNet++ on ScanObjectNN lightweight. Top row: correctly classified examples. Bottom row: misclassified examples. GT and Pred denote the held-out label indices used by the dataset release.

6.4 Comparison with Published Results

To satisfy the course requirement of comparing with the papers, we compare our local results against numbers explicitly reported in the reference papers available in the repository. These comparisons are informative but not perfectly apples-to-apples because our local settings are lighter: fewer input points, CPU execution, shorter training on ScanObjectNN, and reduced subset usage for the lightweight runs.

Table 4: Comparison between our local results and published reference numbers. Negative gap means our local result is lower than the paper result.

Experiment	Our Acc. (%)	Reference Acc. (%)	Gap (pts)	Reference source
PointNet++ on ModelNet40 fast	87.24	90.7	-3.46	PointNet++ paper, xyz-only ModelNet40 classification result [2].
DGCNN on ModelNet40 fast	87.03	92.9	-5.87	DGCNN paper, ModelNet40 classification result [3].
PointNet++ on ScanObjectNN lightweight	74.12	77.9	-3.78	ScanObjectNN paper, PointNet++ result on the full PB_T50_RS benchmark [4].
DGCNN on ScanObjectNN lightweight	66.02	78.1	-12.08	ScanObjectNN paper, DGCNN result on the full PB_T50_RS benchmark [4].

Several reasons explain these gaps:

- Our ModelNet40 runs use 512 input points rather than stronger full-paper settings with more points and more optimized training.
- All four reported configs use CPU runtime, which limited the practical training budget.
- The two ScanObjectNN lightweight runs use only 4,096 training and 1,024 test samples instead of the full benchmark split.
- The lightweight ScanObjectNN experiments use only 6 epochs, which is far from the "train to convergence" setup used in the ScanObjectNN benchmark paper.
- Implementation and hyperparameter details differ from the original public training environments and paper setups.

6.5 Analysis on the Second Dataset

The second dataset analysis is one of the most important parts of this project. Compared with ModelNet40, ScanObjectNN is more difficult for both conceptual and experimental reasons.

- ModelNet40 is synthetic and clean, while ScanObjectNN comes from real scanned scenes.
- ScanObjectNN contains clutter, partial observations, and noisier geometry.
- The selected `pb_t50_rs` variant is specifically designed to be hard because translation, rotation, and scale perturbations increase difficulty.

Our local results follow this expected trend. Even though the settings are not directly matched across datasets, the second dataset is clearly harder to optimize. More importantly, the ScanObjectNN paper shows that even full-scale models drop from around 90% on ModelNet40 to the high-70s on the hardest ScanObjectNN variant [4]. This confirms that the difficulty increase is not just caused by our local compute budget; it is also a property of the dataset itself.

Under our *lightweight* ScanObjectNN setting, PointNet++ outperforms DGCNN by a clear margin. We do **not** interpret this as a universal conclusion that PointNet++ is better than DGCNN on ScanObjectNN in general, because the published full-benchmark numbers in the ScanObjectNN paper show DGCNN slightly ahead of PointNet++ on PB_T50_RS. A more careful interpretation is that under our local reduced-subset and short-training setup, PointNet++ was more stable and DGCNN appears to have been more affected by limited optimization budget.

7 Limitations and Failure Analysis

This project has several important limitations, and stating them clearly is necessary for an honest report.

7.1 Compute and Scope Limitations

The strongest limitation is compute budget. The reported configs were designed to be practical on local CPU execution. This affected the number of epochs, the input point count, and especially the ScanObjectNN experiments. As a result, our paper comparison is valuable for trend analysis, but it is not a strict reproduction of the full published training setup.

7.2 Lightweight ScanObjectNN Is Not a Full Benchmark Run

The ScanObjectNN results in this report come from the unified *lightweight* setting. This setting uses a reduced subset of the local data and only 6 epochs. Therefore:

- it is suitable for controlled local comparison and course-project analysis;
- it is **not** sufficient to claim full reproduction of the ScanObjectNN benchmark numbers;
- it should not be mixed with full-benchmark numbers in a misleading way.

7.3 Failure Analysis

A full per-sample failure benchmark was not collected across all experiments, so most of our failure analysis is still based on dataset properties, training curves, and comparison with the benchmark literature rather than on an exhaustive manually curated gallery.

After preparing the final report draft, we added a small qualitative supplement based on the available PointNet++ checkpoint for the ScanObjectNN lightweight run. The examples in Figure 5 improve the evidence level of this section, but they still do not constitute a full failure benchmark across all experiments.

The most likely failure sources are:

- **Partial observations**: missing object parts reduce the reliability of local geometry.
- **Background clutter**: nearby scene points can distract the classifier from the target object.
- **Non-uniform density and noise**: real scans are less regular than CAD objects.
- **Undertraining on the harder dataset**: the DGCNN lightweight curve suggests that 6 epochs were not enough for stable convergence on ScanObjectNN.

In short, the main failure pattern is not a single bug but a combination of harder real-world geometry and a lightweight training budget.

8 Conclusion and Future Work

This course project built a reproducible benchmark pipeline for 3D point cloud classification using PointNet++ and DGCNN. The codebase supports dataset loading, training, evaluation, logging, checkpointing, and curve visualization in a modular PyTorch structure. On ModelNet40, the two methods achieved very similar performance under matched fast settings. On ScanObjectNN, our lightweight experiments confirmed that the second dataset is harder and more sensitive to training budget, and that honest reporting of subset usage is essential.

The project satisfies the main basic requirements of the course: project introduction, input/output definition, environment setup, pipeline understanding, model training, comparison with reference papers, second-dataset analysis, drawback analysis, and comparison with another state-of-the-art method. However, we do not claim a completed advanced novel extension.

Future work can extend this project in several realistic directions:

- run full ScanObjectNN experiments on the complete split and for more epochs;
- train with GPU resources and stronger hyperparameter tuning;
- test higher input point counts and normals where appropriate;
- add confusion matrices and qualitative failure-case visualization;
- explore a genuine algorithmic extension, such as improved PointNet++ variants or robustness-oriented training, and evaluate it honestly as an advanced component.

9 Project Organization and Contributions

This project was completed as a collaborative effort, with each member taking primary ownership of one major area while also contributing to shared tasks such as experiment support, result checking, analysis, report writing, and presentation preparation.

- **ZHUANG Yi (58800510, leader)** coordinated the project schedule and overall integration, led the PointNet++ line of work, prepared and executed the PointNet++ experiments on ModelNet40 and ScanObjectNN, and consolidated the final report structure and project artifacts.
- **ZHU Chunjin (59845469)** led the DGCNN line of work, including DGCNN implementation adaptation, configuration, experiment setup, debugging, and the preparation of comparison content between DGCNN and the PointNet++ baseline.
- **Xia Junxia (59326842)** provided cross-module support across the project, including dataset preparation assistance, result organization, log and figure checking, analysis support for the second-dataset discussion, and help with polishing the report and presentation materials.

Although responsibilities were differentiated, the final benchmark pipeline, result interpretation, and presentation materials were completed jointly through iterative discussion and mutual support across the three work streams. This balance of differentiated ownership and shared execution better reflects the actual workflow of a three-person graduate course project.

References

[1] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. In *CVPR*, 2017.

[2] Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. In *NeurIPS*, 2017.

[3] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. Dynamic Graph CNN for Learning on Point Clouds. *ACM Transactions on Graphics*, 2019.

[4] Mikaela Angelina Uy, Quang-Hieu Pham, Binh-Son Hua, Duc Thanh Nguyen, and Sai-Kit Yeung. Revisiting Point Cloud Classification: A New Benchmark Dataset and Classification Model on Real-World Data. In *ICCV*, 2019.

[5] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3D ShapeNets: A Deep Representation for Volumetric Shapes. In *CVPR*, 2015.